1. O que é 2. As 5 camadas 3. O dia a dia 4. Mudanças planejadas 5. Auditoria A/B

6. Várias IAs 7. Motor de Estrutura 8. Roteador de modelos 8b. Painel na tela

8c. Governança e wizard 9. Templates 10. Instalar 11. Atualizar 12. Problemas comuns

13. Glossário

Alternar tema claro/escuro

Agentic OS — Guia Completo

Um jeito de organizar os arquivos do seu projeto para que a IA (Claude, Codex, Gemini...) lembre o que já foi feito, siga regras, trabalhe com especialistas e use o modelo certo para cada tarefa.

Plugin para Claude Code e Codex | Versão 1.12.0 | Funciona em qualquer projeto | Tudo opcional e por camadas

ENTENDER

1. O que é e por que existe

🧠 **A analogia**

Usar uma IA num projeto é como contratar um **funcionário brilhante que perde a memória toda noite**. Todo dia você explica de novo o que é a empresa, o que já foi feito, o que deu errado e o que fazer agora. Você perde tempo, ele erra por falta de histórico e cada explicação custa dinheiro (tokens).

O **Agentic OS não é um programa**. É um **método de organização de pastas e arquivos** que monta o "escritório" desse funcionário: crachá com as regras, arquivo de conhecimento, diário

de bordo, caderno de lições, equipe de especialistas e regras de segurança. A IA abre o projeto, lê esses arquivos e **continua de onde parou**.

⏱️

Sem "partida a frio"

A IA não começa do zero a cada sessão.

💰

Menos tokens

Lê só o que precisa, na hora certa.

📓

Memória que cresce

Cada sessão deixa registro e aprendizados.

🛡️

Regras claras

O que exige sua aprovação fica escrito.

O plugin traz a **skill** `agentic-os` (as instruções que a IA segue para montar ou auditar essa organização), **templates** prontos, **comandos** (`/status` , `/wrapup` ...) e camadas opcionais: várias IAs no mesmo projeto, organização do código e o **roteador de modelos**.

ENTENDER

2. As 5 camadas (o escritório)

Toda instalação do Agentic OS tem estas cinco peças. Cada uma é uma pasta ou arquivo comum no seu projeto.

NO ESCRITÓRIO	NO PROJETO	PARA QUE SERVE
🪪 Crachá e manual	`CLAUDE.md` ou `AGENTS.md`	Quem a IA é, regras de trabalho, onde está cada coisa. É lido **automaticamente** em toda sessão, então é curto (até ~150–200 linhas).
🗂️ Arquivo por assunto	`context/`	Conhecimento do projeto: produto, arquitetura, stack. Projetos grandes podem ter uma subpasta por módulo (`context/auth/` , `context/billing/`).

📔 Diário de bordo	`memory/history/`	Um registro por sessão: o que foi feito, decisões e pendências.
📝 Caderno de lições	`memory/learnings/`	Erros que já aconteceram e como foram resolvidos. A IA consulta antes de investigar um bug.
👥 Equipe de especialistas	`workers/`	"Programador", "revisor", "arquiteto"... Cada um com papel, função, contexto que carrega e formato da resposta.
🚦 Regras de segurança	`automation/`	`guardrails.md` (o que pede sua aprovação: apagar, publicar, APIs pagas) e `evaluation.json` (critérios de qualidade antes de entregar).

Peças de apoio

`.claude/commands/` (os comandos), `changes/` (mudanças planejadas), `AGENTIC-OS.md` (resumo rápido do sistema) e um **gancho de encerramento** que, ao fechar o Claude Code, deixa um lembrete em `memory/history/` para você rodar `/wrapup`.

ENTENDER

3. O dia a dia

1 · Chegar
"vamos começar": a IA lê o diário e diz onde parou.

2 · Situação
`/status` : resumo e as 3 pendências principais.

3 · Trabalhar
Pedido pequeno: fale normalmente.
Grande: `/propose` .

4 · Fechar
`/wrapup` : grava diário e lições.

⚠️ **O passo mais importante:** `/wrapup`

É ele que dá memória ao sistema. Sem ele, a próxima sessão não sabe o que aconteceu hoje.

Comandos

COMANDO	O QUE FAZ
`/status`	Branch atual, última sessão, estado das funcionalidades, top 3 pendências.
`/propose <nome>`	Cria uma mudança planejada em `changes/<nome>/` (proposta + lista de tarefas). Sem nome, lista as mudanças ativas.
`/worker <nome>`	Ativa um especialista e executa as tarefas da mudança ativa. Sem nome, lista os especialistas.
`/wrapup`	Fecha a sessão: diário, lições, arquiva a mudança concluída e atualiza o `context/`.
`/conform`	Audita se o projeto segue o padrão (e corrige com sua aprovação).
`/handoff`	Lê ou grava a "passagem de turno" entre IAs (projetos multi-provedor).
`/structure`	Monta ou audita a organização das pastas do código.
`/router-update`	Atualiza o roteador de modelos no projeto (só onde ele está instalado).

No Codex e no Gemini, os mesmos comandos funcionam pedindo em linguagem natural ("faz o wrapup"): a lógica de cada um está escrita em `automation/procedures/`, igual para todas as IAs.

ENTENDER

4. Mudanças planejadas (`/propose` → `/worker` → `/wrapup`)

A analogia

Antes de reformar uma casa, o mestre de obras mostra a planta e a lista de etapas. Você aprova, e só então a obra começa.

1. `/propose adicionar-pagamento-pix`: a IA cria `changes/adicionar-pagamento-pix/` com `proposal.md` (o quê e por quê), `tasks.md` (passos com caixinhas `[ ]`) e, se for complexo, `design.md`.
2. **Você lê e aprova**: ajusta o que quiser antes de começar.
3. `/worker developer`: o especialista executa as tarefas, marcando `[x]` conforme termina.
4. `/wrapup`: a mudança vai para `changes/archive/` e o que mudou entra no `context/`.

Reorganizar um projeto existente (mover pastas, juntar documentos) só acontece dentro de uma mudança aprovada (`/propose reorganize-<alvo>`), com regras rígidas: nada é apagado (vai para `_quarantine/`), referências são checadas e cada movimento fica registrado para desfazer.

FUNCIONALIDADES

5. Montar e auditar: o loop A/B

Ao pedir `/agentic-os` ("analisa este projeto" ou "monta um projeto novo") ou `/conform`, o plugin trabalha com dois papéis e você no meio:

Agente A · Auditor
Só lê. Compara com o padrão e marca cada item: CONFORME, FALTA ou DRIFT (fora do padrão).

Você · Aprovação
Aprova tudo ou item por item.

Agente B · Executor
Faz só o que foi aprovado, nada além.

Agente A · Confere
Verifica no disco. Passou → fim. Não passou → corrige de novo (máx. 3 voltas).

Projeto existente (Modo A)

Regra de ouro: só cria. Nunca move, renomeia ou apaga o que já existe. Conteúdo espalhado vira referência (`@arquivo`), sem duplicar.

Projeto novo (Modo B)

Pergunta tipo de projeto, stack, público e objetivo, e preenche os arquivos com conteúdo real, não com modelo vazio.

FUNCIONALIDADES · OPCIONAL

6. Várias IAs no mesmo projeto

> 🔁 **A analogia**
>
> Um hospital com troca de plantão: quem sai deixa um bilhete curto para quem entra. Ninguém recomeça o atendimento do zero.

- **Um cérebro só:** `AGENTS.md` é o arquivo principal. `CLAUDE.md` e `GEMINI.md` só apontam para ele (`@AGENTS.md`), então ninguém lê versões diferentes.
- **Passagem de turno:** `memory/handoff.md` tem no máximo 20 linhas, com qual IA trabalhou, quando, a mudança ativa e o próximo passo. O "onde parei" é sempre a primeira caixinha `[ ]` do `tasks.md`, nunca copiado à mão.
- **Âncora no git:** ao fechar cada tarefa grava-se o `commit`, então "o que aconteceu" vem do histórico real.
- **Orquestração (opcional): você é o chefe final**. Uma IA coordena e propõe lotes de 3 a 5 tarefas; as outras só executam até o **Limite aprovado**, que só avança com sua aprovação registrada.

FUNCIONALIDADES · OPCIONAL

7. Motor de Estrutura (organização do código)

As camadas anteriores organizam o **cérebro da IA**; o Motor organiza as **pastas do código** da sua aplicação, de forma verificável. Comando: `/structure`.

- **Perfis:** `next-app` (Next.js App Router, com componentes, ações e acesso a dados junto de cada rota) e `generico` (organização por funcionalidade).
- **Uma regra para todas as IAs:** as regras são escritas uma vez e convertidas para o formato do Cursor, Copilot, Codex e Gemini, com checagem automática para nunca divergirem.
- **Projeto existente começa no "Tier 0":** só instala as regras; o código não é tocado e o que for novo já nasce organizado. Migrar um módulo é opcional, via `/propose migrate-<módulo>`, e só vale se build e testes passarem.

FUNCIONALIDADES · OPCIONAL · CLAUDE CODE

8. Roteador de modelos (com Jev)

A analogia: uma clínica com recepção

Você chega com um problema. A **triagem** (o Jev) avalia em menos de meio segundo. A **recepcionista** (Haiku, a IA mais barata) não atende ninguém: só encaminha. Coisa simples vai para o clínico rápido; trabalho comum, para o profissional padrão; caso difícil, para o especialista. Você recebe só o resumo do atendimento.

rapido **Haiku**

Barato e rápido

- Trocar texto, cor, renomear
- Perguntas só de leitura: "o que faz este arquivo?", "como funciona o login?"

padrao **Sonnet**

Equilibrado

- Tela nova, endpoint, testes
- Campo novo num cadastro
- Refatoração com escopo claro

profundo **Opus**

O mais capaz

- Bug sem causa conhecida
- Mexer em autenticação, dados, segurança
- Mudança que atravessa o projeto

O que acontece quando você manda uma mensagem

1
Você digita.

2 · Gancho
Antes de a IA ler, um script (`route.mjs`)

3 · Decisão
Regras rápidas ou o Jev escolhem o

4 · Recepção
O Haiku entrega ao subagente indicado.

	entra em ação.	executor.	

5 · Resumo
Feito, verificações e pendências.

Ordem da decisão

1. **Passa direto (sem roteamento):** `/comandos`, `!comando` (modo terminal do Claude Code) e avisos internos do próprio Claude Code (retorno de subagente, notificações).
2. **Você escolheu:** `nivel:avancado conserte o login` (ou `basico`, `intermediario`) força o nível sem chamar o Jev.
3. **Regras fixas** (`rules`): padrões de texto que você define vão direto a um nível, sem chamar o Jev.
4. **Continuação:** "sim", "pode seguir", "opção 2" repetem o nível da tarefa anterior, sem chamar o Jev.
5. **Jev:** recebe seu pedido, o pedido anterior e um resumo do projeto, e diz **qual executor** é o menor que faz bem o trabalho, com a probabilidade de cada um.
6. **Rede de segurança:** se houver chance relevante de precisar de mais força (padrão ≥ 50% somando os níveis acima, profundo ≥ 35%), sobe de nível. Se o Jev falhar (sem chave, fora do ar, demora acima de 5 s), vai para o **profundo**.

O que é o Jev

Uma IA da TypeSafe feita para **respostas curtas e objetivas**, e não para conversar: devolve uma escolha e a probabilidade, por exemplo `padrao 0.98`. Roda em ~0,3–0,5 s por pedido. Ele não lê seus arquivos nem programa; só ajuda a escolher. A decisão final é sempre aplicada pelo código, de forma previsível.

Resultado real (teste com um projeto de barbearia)

PEDIDO	FOI PARA
Troque o texto do botão "Agendar"	rapido 100%

Como funciona o login do painel? · Explique o pagamento com Stripe	rapido 99%
Crie a tela de histórico de agendamentos	padrao 99%
Adicione o campo telefone no cadastro	padrao 80%
Dois clientes marcam o mesmo horário, descubra por quê	profundo 77%
Migre a autenticação de NextAuth para Clerk	profundo 100%
"sim, pode seguir" (após a tarefa anterior)	profundo continuação

Como acompanhar

- **Barra de status** (rodapé do Claude Code): `roteador: jev 0.98 → padrao (Sonnet)`. Outras formas: `manual →` (você forçou), `continua →`, `regra:<nome> →`, `erro → profundo`.
- **Registro:** `.claude/router/decisions.jsonl`, uma linha por pedido com horário, início da mensagem (80 caracteres, nunca o texto inteiro), escolha, probabilidades, motivo e tempo do Jev.

Ajustes (`.claude/router/config.json`)

CHAVE	PARA QUÊ
`agent_criteria`	Descrição, exemplos e exclusões de cada executor: é o que o Jev lê para escolher. É o principal ajuste fino.
`policy`	Quando subir de nível (`probabilidades`, padrão em projetos novos, ou `confianca`).
`rules`	Atalhos por padrão de texto, ex.: tudo que fala de "migration" vai para o profundo.
`continuacao`	Quais respostas curtas herdam o nível anterior.
`tiers`	Qual modelo cada executor usa. Depois de mudar, rode `node .claude/router/emit-agents.mjs`.

`fallback_tier` · `max_escalations`	Para onde ir se o Jev falhar · quantas vezes um executor pode passar a tarefa para o nível acima.
`decision_mode`	`jev` (o Jev escolhe) ou `regras` (o Jev só classifica e o código decide pelos limites de risco e confiança).

Limitações honestas

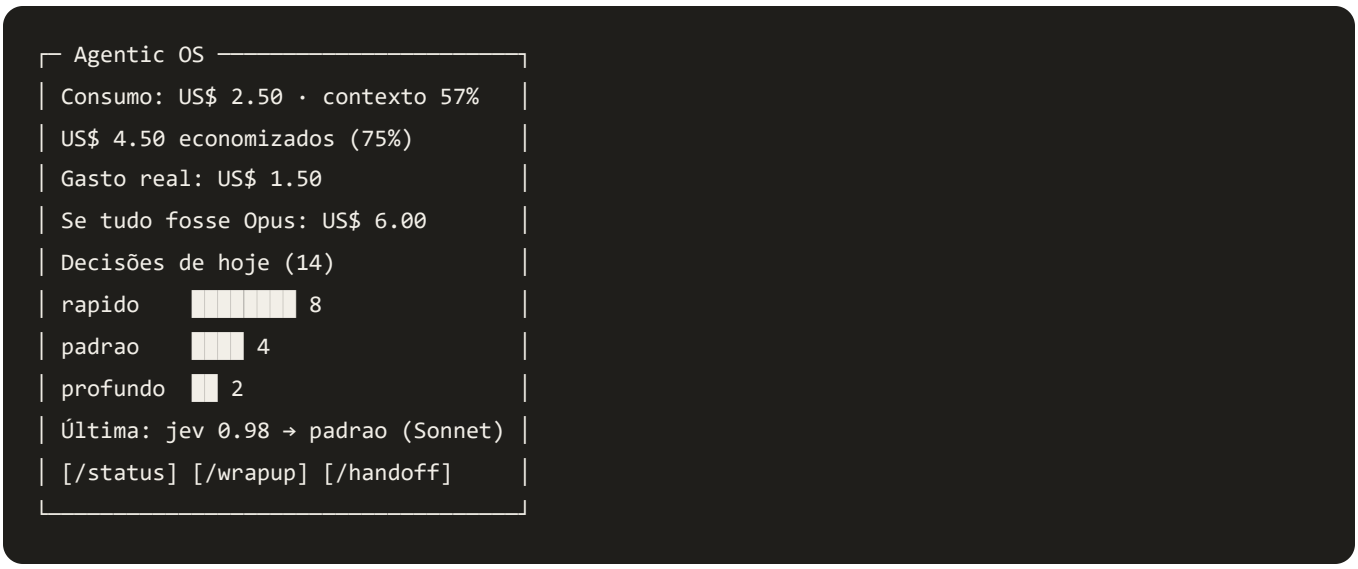
A recepção (Haiku) é orientada a sempre delegar, mas isso não é garantido 100%: vale olhar o registro nos primeiros dias. Cada mensagem soma ~0,5 s e alguns centavos do Jev. Seu pedido e um resumo do projeto vão para os servidores da TypeSafe, então não coloque senhas no `CLAUDE.md` nem nas mensagens.

FUNCIONALIDADES · OPCIONAL · CLAUDE CODE

8b. Painel: economia e atalhos na tela

A analogia: o painel do carro

Assim como o painel mostra o consumo enquanto você dirige, este painel fica ao lado do chat e mostra quanto o roteador está economizando, para onde foram os pedidos do dia e três botões para os comandos que você mais usa.



- **Consumo desta sessão:** sempre visível, com ou sem roteador: quanto a sessão já custou em US$ (o mesmo número do `/cost`) e quanto do contexto está em uso. Atualiza sozinho.
- **Economia:** ao terminar cada subagente, em primeiro ou segundo plano, o painel lê os tokens e o modelo no registro dele e calcula o gasto real e quanto custaria se tudo tivesse ido para o Opus. A diferença, menos o custo do Jev, é a economia.
- **Decisões de hoje:** lidas do registro do roteador. O painel tem o mesmo formato em todo projeto: sem o roteador, as seções aparecem vazias, com um aviso.
- **Botões:** rodam `/status`, `/wrapup` e `/handoff` como se você tivesse digitado.

Instalar (é um plugin separado, opcional):

```
claude plugin install agentic-os-painel@agentic-os
```

Reinicie o Claude Code: o painel abre sozinho, e `/painel` reabre. Funciona no VS Code, no app Desktop e no terminal largo (a partir de ~110 colunas). Precisa de um Claude Code com suporte a mods.

> **É uma estimativa**
> Os preços ficam numa tabela dentro do plugin e precisam de atualização quando a Anthropic mudar. O custo da recepção (Haiku) não entra, e a economia zera a cada sessão.

FUNCIONALIDADES · OPCIONAL

8c. Governança GitHub e wizard

> 🚥 **A analogia: a esteira com portas trancadas**
> Cada pedido (cartão) anda numa esteira própria (uma branch). Para entrar no estoque oficial (a `main`), a porta só abre com o crachá do analista (a aprovação no GitHub), e se alguém mexer na peça depois do crachá, a porta tranca de novo.

O wizard

Ao pedir para configurar o projeto, o `/agentic-os` agora conversa em etapas: **uma pergunta por tela**, com 2 a 4 opções (a recomendada vem primeiro), botão de voltar e de pular. Se a conversa cair, ele retoma de onde parou. Os avisos de custo aparecem logo no começo, antes de configurar qualquer coisa.

O que a governança faz

- **Ficha do projeto** (`.agentic-os/governanca.json`): repositório, Project, colunas, analista(s) e ambientes. Nunca guarda senha ou chave.
- `/cartao <ID>` **:** pega o cartão que está em "A Fazer", cria a branch `feature/{ID}-{descrição}` (ou `fix/`), cria o arquivo `docs/tasks/TASK-{ID}-{descrição}.md` e move o cartão para "Em andamento".
- `/cartao pr <ID>` **:** envia a branch, abre o PR para a `main` citando o cartão e comenta no cartão.
- `main` **protegida:** só entra por PR, com aprovação do analista; commit novo derruba a aprovação; vale até para administradores; sem apagar nem reescrever histórico.
- **Ambientes** `hom` **e** `prd` **:** o `prd` exige aprovação do analista.

Supervisor: nota de 0 a 10

Pense num **fiscal de obra**: quem construiu não se fiscaliza. Antes do PR, o `/cartao revisar <ID>` chama um fiscal independente que confere cada item da lista de aceite do cartão e só carimba com prova (arquivo e linha, ou comando e resultado). O que dá para medir com régua é medido por um script, sem opinião de IA.

- **O script mede:** os testes do projeto passam, nenhum segredo no código da branch, trabalho na branch certa e o arquivo TASK existe. O script também calcula a nota (itens atendidos ÷ total, de 0 a 10).
- **O Supervisor julga:** regras do projeto respeitadas, documentação atualizada e cada "Critério de aceite" do cartão. Sem prova, o item conta como não atendido.
- **Só o 10 passa direto** para o `/cartao pr` . Abaixo disso, o agente corrige só o que faltou e revisa de novo.
- **Na 3ª nota abaixo de 10, você decide:** seguir com a nota atual ou pedir uma nova análise. Se você liberar, o PR mostra um aviso ⚠️ com a nota e o motivo.
- **A nota vale para um commit.** Mexeu no código depois, precisa revisar de novo.
- **Dica:** se aparecer erro de "árvore suja", commite o que faz parte do cartão ou ponha pastas geradas pelos testes (como `coverage/`) no `.gitignore` .

Homologação e assinatura

Pense no **gerente de obra e no dono da obra**. O gerente (o agente) coloca a peça pronta na sala de demonstração (o ambiente HOM), **um cliente por vez**, e avisa o dono (o analista). Quem diz "aprovado" ou "refaça" é só o dono, por escrito no PR. O gerente apenas lê e anota; nunca assina por ele.

- **Antes de tudo:** a ficha ganha os comandos do HOM (`deploy_hom`, `migracao_hom`, `testes_hom`) e o link (`url_hom`). Senhas e chaves nunca ficam na ficha.
- `/cartao hom <ID>`**:** publica no HOM o commit que o Supervisor revisou. Se o HOM está com outro cartão, ele avisa e espera; furar a fila só com o seu "sim" e um motivo. Migração de banco também só roda com o seu "sim". Se não houver comando de deploy, é manual: você publica e confirma. Arquivo novo não ignorado trava o comando, para o HOM nunca receber algo diferente do que foi revisado.
- **Depois de publicar:** o cartão vai para "Em validação" e o PR recebe um comentário com o link do HOM e a lista do que testar. Nada mais é commitado na branch (um commit novo derrubaria a aprovação).
- **O que o analista faz:** abre o PR e escolhe **Approve** (aprova) ou **Request changes** (reprova, explicando o que falhou).
- `/cartao assinatura <ID>`**:** lê a review do analista (a última de cada analista; a conta do agente não vale; precisa ser do commit publicado). Aprovado: a assinatura entra no corpo do PR e o cartão vai para "Aprovado". Reprovado: o motivo vai para o arquivo TASK e o cartão volta para "Em andamento", para corrigir e passar de novo por revisar, pr e hom.
- **Se outro cartão tomar o HOM** antes da assinatura, a homologação pendente deixa de valer: é preciso rodar `/cartao hom` de novo.
- **Limite do plano gratuito:** em repositório privado no plano gratuito, o GitHub registra a review mas **não obriga** o merge a esperar por ela. O `/cartao merge` exige a assinatura; um merge manual pela interface continua possível.
- `/cartao merge <ID>`**:** integra na `main` como um cartório, que só registra depois de conferir a assinatura na hora. O plugin relê a review do analista no GitHub (não confia na cópia local), confere que ela é do último commit do PR, que os checks passaram e que não há conflito. O merge é em squash, com a assinatura na mensagem do commit; a branch do cartão é apagada e a issue é fechada. Se a assinatura foi pelo chat, o merge só acontece quando você disser, por iniciativa própria, que quer integrar. O cartão fica em "Aprovado"; a publicação em PRD vem depois.

Identidade do agente

Pense em um **crachá próprio para o agente**. Se o agente usa o seu crachá, o GitHub não deixa você aprovar o que "você mesmo" abriu, e uma aprovação sua poderia ter sido dada pelo agente. Com crachá próprio, o analista assina com o dele e a portaria (o GitHub) sabe distinguir os dois.

- **Três modos** (por projeto, na ficha): **nenhuma** (o agente usa o login do `gh`, como antes), **token** (uma conta-robô com token numa variável de ambiente) e **app** (um GitHub App: o plugin gera o token de 1 hora com a chave privada). O segredo nunca fica na ficha nem no git; a ficha guarda só o nome da variável.
- **Sem crachá separado,** o `/cartao hom` e o `/cartao assinatura` param se o agente for um dos analistas (`IDENTIDADE_IGUAL_ANALISTA`).
- **Aprovação pelo chat (opcional):** um "livro de visitas". Registra a frase do usuário como "chat, sem prova do GitHub": é **prova fraca**, porque ninguém garante quem escreveu. Se já existe uma review no PR, ela vence.
- **Dica:** deixe o `gh` da máquina logado só com a conta do agente; se estiver logado como analista, a assinatura sai com o aviso ⚠️.
- **Limites:** testado só com um GitHub simulado, ainda não no real; em modo token ou app, prefira um Project de organização, porque esses tokens podem não escrever em Project de conta pessoal.

O que você precisa

- O `gh` (GitHub CLI) instalado e logado, com a permissão de Projects (`gh auth refresh -s project`).
- O analista com **conta GitHub própria**: o GitHub não deixa quem abriu o PR aprová-lo. O wizard convida a conta para o repositório.

Pode custar

Em repositório **privado no plano gratuito** do GitHub, a trava da `main` (PR + aprovação do analista) e a aprovação do ambiente `prd` **não funcionam**: o GitHub bloqueia as duas (confirmado em teste real). O wizard avisa no início e oferece três caminhos: tornar o repositório público, assinar plano pago ou seguir com proteção parcial — sem trava na `main`, a disciplina fica só nas regras do agente (fica registrado na ficha).

Supervisor: cada revisão de cartão usa o modelo Opus (até 3 por rodada), mesmo com o roteador Jev ligado.

Travas que não se desligam

O script nunca faz push nem merge na branch principal, nunca aprova PR, nunca apaga nada e só **endurece** a proteção: se a regra atual já for mais forte, ela é mantida. Detalhes em `docs/GOVERNANCA.md`.

FUNCIONALIDADES

9. Templates prontos

TEMPLATE	PARA
`A-generico`	Conteúdo, marketing, projetos não técnicos.
`B-saas-n8n`	SaaS com automações n8n (WhatsApp, Telegram...).
`C-claude-integrado`	SaaS usando Claude Code com Superpowers.
`D-multi-provedor`	Várias IAs no mesmo cérebro, com passagem de turno e orquestração.
`E-roteador`	Camada do roteador de modelos (instalada pelo instalador próprio, sobre qualquer projeto).

Especialistas sugeridos por tipo de projeto: SaaS (*developer, workflow-designer, arquiteto, qa*), conteúdo (*roteirista, pesquisador, analista*), consultoria (*consultor, gestor, documentador*). Use só os que fazem sentido.

USAR

10. Instalar

Passo 1: instalar o plugin (uma vez por computador)

Claude Code, no terminal:

```
claude plugin marketplace add rasecagapito/agenticOS
claude plugin install agentic-os@agentic-os
```

Ou dentro do Claude Code: `/plugin marketplace add rasecagapito/agenticOS` e `/plugin install agentic-os@agentic-os`. Reinicie o Claude Code depois.

Codex: `codex plugin marketplace add rasecagapito/agenticOS`, depois abra **Plugins**, instale **Agentic OS** e inicie uma nova tarefa.

Gemini e outras IAs: não precisam de plugin. Usam os arquivos que o Agentic OS cria no projeto (`AGENTS.md`, `GEMINI.md`, `automation/procedures/`).

Passo 2: montar o Agentic OS num projeto

1. Abra o Claude Code na pasta do projeto.
2. Digite `/agentic-os` e peça **"analisa este projeto"** (se já existe) ou **"monta um projeto novo"**.
3. Leia a tabela do que falta e aprove. Nada seu é apagado nem movido.
4. Para várias IAs, peça também "ativa o multi-provedor".

Passo 3 (opcional): ligar o roteador de modelos

> ⚠️ **Leia antes: sem a chave do Jev, tudo vai para o Opus**
>
> O roteador depende de uma chave **sua** do Jev. Se ela não estiver configurada, ou estiver errada, **todos os pedidos vão para o Opus**, o modelo mais caro, até você corrigir. Tenha a chave em mãos antes de instalar. Se for instalar sem ela, defina `"fallback_sem_chave": "intermediario"` no `config.json` para usar o Sonnet enquanto isso.

1. **Sua chave do Jev:** o plugin não traz chave nenhuma, e cada pessoa usa a própria. Crie a sua em console.typesafe.ai/keys e guarde só em variável de ambiente, nunca em arquivo versionado:

   ```
   setx TYPESAFE_API_KEY "sua-chave"        # Windows (reabra terminal e VS Code)
   export TYPESAFE_API_KEY=sua-chave        # macOS / Linux
   ```

Na instalação, o instalador **testa a chave** e diz se está OK, recusada ou faltando.

2. **Pré-visualizar** o que vai mudar (não altera nada):

```
node "<plugin>/scripts/install-router.mjs" "C:/caminho/do/projeto" --dry-run
```

`<plugin>` é a pasta do plugin instalado, ex.: `C:/Users/<você>/.claude/plugins/cache/agentic-os/agentic-os/1.12.0` (ou um clone do repositório).

3. **Instalar:** o mesmo comando sem `--dry-run`.

4. **Reiniciar** o Claude Code no projeto.

O que o instalador muda no projeto

Cria `.claude/router/` e os subagentes `rapido`, `padrao` e `profundo`; liga o gancho; **troca o modelo principal do projeto para Haiku** (a recepção); configura a barra de status (somando à sua, se já tiver uma); acrescenta 2 linhas ao `CLAUDE.md`; ignora o registro no git. Não apaga hooks, permissões nem arquivos existentes. Pode rodar de novo sem duplicar nada.

USAR

11. Atualizar

O plugin (em cada computador)

```
claude plugin marketplace update agentic-os
claude plugin update agentic-os@agentic-os
```

O plugin vem direto da `main` do GitHub: quando sai versão nova, esses dois comandos já trazem. Reinicie o Claude Code.

Projetos com instalação própria

Se um projeto instalou o plugin só para ele (escopo de projeto), rode o `claude plugin update agentic-os@agentic-os` **dentro da pasta desse projeto**.

O roteador (em cada projeto que o usa)

Depois de atualizar o plugin, dentro do projeto:

```
/router-update
```

- Atualiza scripts, procedimento e subagentes para a versão nova.
- **Preserva** seu `config.json` e o registro de decisões.
- Novas opções não são ligadas sozinhas, para não mudar o comportamento sem você saber. O instalador lista as disponíveis e você ativa no `config.json`.
- Melhorias nos critérios (`agent_criteria`) também não são copiadas, porque seriam as suas. Para usar os critérios novos, copie o bloco do template (`template/E-roteador/.claude/router/config.json`).

Histórico recente

VERSÃO	PRINCIPAL NOVIDADE
1.12.0	Merge exigindo a assinatura: `/cartao merge` relê a review do analista e só integra o commit assinado (squash com a assinatura na mensagem).
1.11.0	HOM e assinatura: `/cartao hom` publica o commit revisado no HOM (um cartão por vez) e `/cartao assinatura` lê o Approve ou Request changes do analista no PR e registra.
1.10.0	Supervisor: `/cartao revisar` dá nota de 0 a 10 com prova; o PR só abre com nota 10 ou liberação do usuário (aviso ⚠️ no PR).
1.9.0	Wizard de configuração (uma pergunta por tela) e camada opcional de Governança GitHub: cartão, branch, PR e `main` protegida com aprovação do analista.
1.8.7	Painel 1.2.1: mesmo formato em todos os projetos, com ou sem roteador.
1.8.6	Painel 1.2.0: a economia passa a contar subagentes em segundo plano.
1.8.5	Correção: pedidos enviados com o Chrome conectado voltam a passar pelo Jev.

1.8.4	Correção: o roteador não roda mais duas vezes em máquinas com gancho global (ex.: Agente de Cargas).
1.8.3	Correção: a versão do plugin agentic-os passa a acompanhar a release (projetos mostravam 1.8.0).
1.8.2	Painel 1.1.0: consumo da sessão (US$ do /cost e % do contexto) sempre visível, mesmo sem roteador.
1.8.0	Painel opcional no Claude Code (`agentic-os-painel`): economia do Jev, decisões do dia e botões /status, /wrapup e /handoff.
1.7.3	Aviso logo no início da configuração: sem a chave do Jev, todos os pedidos vão para o Opus.
1.7.2	Cada pessoa usa a própria chave do Jev: o instalador testa a chave e a conversa avisa (1x por sessão) se ela faltar ou for recusada; `fallback_sem_chave` opcional.
1.7.1	Perguntas só de leitura vão para o `rapido`, calibrado com o Jev real (11/11).
1.7.0	Jev no padrão TypeSafe: "menor executor que faz bem", política por probabilidades, regras, continuação, filtro de avisos internos.
1.6.0	O Jev passa a escolher o subagente; contexto da mensagem anterior.
1.5.0	Roteador de modelos (Jev + Haiku/Sonnet/Opus).
1.4.0	Passagem de turno reforçada e orquestração com você como chefe final.

USAR

12. Problemas comuns

▶ **O `/agentic-os` não mostra as novidades**

▶ **Apareceu "⚠️ Roteador de modelos: ... não definida / recusada"**

▶ **A barra mostra `roteador: erro → profundo`**

▶ **Tudo está indo para o Opus**

▶ **Respondi "sim" e foi para o Haiku**

▶ **A recepção respondeu sozinha em vez de delegar**

▶ **Quero desligar o roteador num projeto**

▶ **Comecei a mensagem com ！ e nada aconteceu**

USAR

13. Glossário

TERMO	SIGNIFICADO
Skill	Conjunto de instruções que a IA carrega para fazer um tipo de tarefa.
Plugin	Pacote instalável que traz skills, comandos e templates para o Claude Code ou Codex.
Token	Unidade de texto que a IA lê e escreve; é o que se paga.
Subagente	IA auxiliar que recebe uma tarefa da sessão principal e devolve o resultado.
Gancho (hook)	Script que o Claude Code roda automaticamente num momento (ao enviar mensagem, ao fechar…).
Drift	Quando algo saiu do padrão ou ficou duplicado e desatualizado.
Handoff	Passagem de turno entre IAs.
Haiku · Sonnet · Opus	Modelos do Claude, do mais barato e rápido ao mais capaz.

Jev	Modelo da TypeSafe que dá respostas objetivas com probabilidade; usado pelo roteador para escolher o executor.